

What CKS Inherits from OIDA Alone: Signed Contradiction Edges and Modeled Ignorance as Standalone Architectural Primitives in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 5 May 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize what CKS inherits from OIDA — one of the two named prior-art lines the source paper positions CKS against — as standalone architectural inheritance with independent operational content, separable from what CKS inherits from Knowledge Objects (KO) and from what CKS contributes freshly. The note follows the integrating-frame treatment of KO/OIDA inheritance and pairs with the parallel KO-only treatment, the two-axis-extension-structure treatment, the architectural-difference-versus-feature-addition treatment, and the multi-human operational-requirements treatment.

Abstract

The CKS pattern positions itself relative to two adjacent prior-art lines, and a parent foundational note (Li 2026, *Inheritance and Extension*) establishes the joint inheritance frame. This note specializes the OIDA half. The source paper §5.2 is precise about what CKS inherits from OIDA: two architectural primitives — *signed contradiction edges* between conflicting Knowledge Objects, and *QUESTION-as-modeled-ignorance* as a first-class representation of missing knowledge. The same section names three architectural axes on which CKS differs from OIDA at the core-theory scope (governance, role/authority schema, AI-as-substrate-mediator). This note states the two-primitive inheritance precisely, identifies what is *not* inherited from OIDA (epistemic-class typology, importance-with-decay, the Knowledge Gravity Engine, AI-as-substrate-client posture), distinguishes the inheritance from four adjacent prior-art frames it is commonly conflated with (paraconsistent logic, Dung-style abstract argumentation, detect-resolve-forget conflict pipelines, pending-resolution treatments of unknowns), names the load-bearing role the inheritance plays for downstream CKS commitments (conflict-as-first-class, substrate-as-source-of-truth, the architectural-difference claim, the two-axis extension structure), enumerates seven failure modes that mis-attribute OIDA inheritance, and provides an operational test for whether a system stands in the OIDA-inheritance relationship the source paper §5.2 actually defends.

1. Why OIDA inheritance needs to be formalized as standalone

The integrating-frame note on KO/OIDA inheritance treats both prior-art lines together because the source paper's positioning move — establishing that CKS is *architecturally different from OIDA on the multi-human axis, not OIDA plus multi-human* — depends on the two inheritance treatments being legible as one continuous argument. The joint frame is correct as far as it goes, but it leaves the OIDA half architecturally underspecified for two specific downstream uses.

The first use is the architectural-difference-versus-feature-addition argument the source paper makes at §5.2 and again at §9.4. That argument requires naming OIDA's contribution narrowly: if OIDA's contribution is overstated, CKS looks like a thin extension of OIDA and the architectural-difference claim weakens; if it is understated, the inheritance the source paper itself acknowledges is obscured and the prior-art posture loses credibility once a reviewer identifies the unacknowledged precedent. The narrow specification — exactly two primitives, contextualized within exactly three differentiation axes — is what holds the architectural-difference claim in tension. A standalone OIDA-inheritance treatment is what makes that tension precise.

The second use is contextualizing the existing operational decomposition of conflict-as-first-class. A separate note (the OIDA-signed-contradiction-edges treatment within the conflict-provenance decomposition) cites OIDA specifically as architectural antecedent for one of the four provenance requirements that conflict-as-first-class places on substrate content. That note formalizes one specific inheritance point. The present note articulates the broader OIDA contribution within which that specific inheritance point sits — the two paired primitives the source paper §5.2 names, of which signed contradiction edges is one and QUESTION-as-modeled-ignorance is the other. The present note does not re-litigate the conflict-provenance treatment; it situates it.

A third motivation is the strategic prior-art posture. Patentable derivations of CKS that focus on substrate-level conflict representation, or on substrate-level missing-knowledge representation, are most defensibly contested when the OIDA inheritance is publicly formalized as standalone — when the territory CKS occupies on the conflict-and-modeled-ignorance axis is already mapped to a specific prior-art primitive by name, and the architectural surplus CKS adds is already characterized as the three differentiation axes operating on top of that primitive.

2. What CKS inherits from OIDA, defined precisely

The source paper §5.2 names two OIDA-specific primitives as architectural antecedents, and a CKS implementation that stands in the OIDA-inheritance relationship inherits both.

(a) Signed contradiction edges as first-class relationship primitive. OIDA represents contradictions between Knowledge Objects as edges that have their own schema presence — an edge type with its own identity, addressing both contradicting objects, persisting in the substrate rather than existing as transient state consumed in a single inference pass. The architectural commitment the primitive realizes is that *the contradiction relationship is itself substrate content*, not a runtime artifact computed over substrate content. CKS inherits this primitive as the architectural antecedent for promoting the contradiction relationship to first-class addressable substrate state — the move the conflict-as-first-class

commitment depends on at the relationship level.

(b) QUESTION-as-modeled-ignorance as first-class missing-knowledge primitive. OIDA represents what the system does not know as substrate content in its own right — a QUESTION object that is present in the substrate, addressable across sessions, distinguishable from the absence of a substrate object. The architectural commitment the primitive realizes is that *modeled ignorance is substrate state, not the absence of substrate state*. CKS inherits this primitive as the architectural antecedent for treating missing knowledge as first-class addressable substrate state on the analogue axis to the one signed contradiction edges occupy.

The two primitives are paired in §5.2's treatment, not selectable. Signed contradiction edges name conflict-as-substrate-state on the contradiction axis; modeled ignorance names the same architectural move on the missing-knowledge axis. The source paper treats them as joint inheritance because the joint commitment — that conflict and missing knowledge can be addressable substrate objects rather than error or pending states — is what the OIDA inheritance contributes architecturally. An implementation that inherits one primitive without the other has not stood in the inheritance relationship the source paper defends; it has inherited half of it.

This two-primitive inheritance is narrow on purpose. The source paper does not attribute typed structured knowledge units to OIDA — that commitment is attributed at §5.2 and §8.2 to Zahn and Chana's Knowledge Objects paper, of which OIDA is described as a downstream architectural family member. Nor does it attribute provenance-field vocabulary to OIDA — provenance vocabulary is attributed at §3.1 and §8.2 to PROV-O / EP-Plan (Naja et al. 2021), and path-retraceability vocabulary to Rajabi & Kafaie (2022). The OIDA inheritance is two primitives, and the parent integrating-frame note's enumeration of OIDA-inherited content reduces to those two primitives at the operational level.

3. What OIDA inheritance does not include

Stating the inheritance precisely requires stating what is not inherited just as precisely, because OIDA carries architectural content beyond the two primitives §5.2 attributes inheritance from, and CKS does not commit to that content.

Not OIDA's epistemic-class typology. OIDA types Knowledge Objects with an epistemic class drawn from a specific four-element vocabulary (fact, hypothesis, decision, question). CKS does not commit to that typology. CKS substrates carry typed content under whatever schemas the deployment authors, and the typology is a deployment choice within the broader human-governed schema-authoring authority.

Not OIDA's importance-score-with-class-specific-decay mechanism. OIDA attaches an importance score with class-specific decay to Knowledge Objects, supporting a runtime maintenance behavior in which content fades according to class-conditioned rules. CKS does not inherit this mechanism. The substrate's persistence and content-evolution behavior is governed by orchestration rules humans author, not by class-conditioned decay.

Not the Knowledge Gravity Engine as deterministic maintenance mechanism. OIDA's Knowledge Gravity Engine is the architectural component that maintains the substrate over time deterministically.

CKS does not inherit a substrate-maintenance mechanism in this sense. Substrate maintenance under CKS is governed at the authority level — exercised at orchestration-rule-authoring time and at direct-override time, per the human-governed commitment — not by a deterministic engine running over substrate state independently of human authority.

Not the AI-as-substrate-client architectural posture. OIDA frames the AI as substrate client, consuming structured memory to answer queries or perform classifications. CKS frames the AI as substrate mediator. This is the third differentiation axis the source paper §5.2 names and is not absence-from-OIDA but explicitly distinct: the inheritance commitment carries no AI-architectural-role inheritance from OIDA, and any system that inherits the two primitives while preserving the substrate-client posture has reproduced OIDA at the AI-role level rather than instantiated CKS.

Not OIDA's single-human scope assumptions. OIDA's architectural treatment is at single-human scope. CKS's multi-human extension is fresh, not OIDA-inherited; the multi-human-axis operational requirements are treated separately. Implementations that read OIDA as silent on the multi-human axis are reading OIDA correctly; CKS's multi-human commitments do not flow from OIDA inheritance.

Not procedural-governance assumptions. OIDA does not specify a governance architecture; it specifies a maintenance architecture. Whatever procedural governance OIDA-deployed substrates may carry — review workflows, approval cycles, audit cycles around the Knowledge Gravity Engine — is not CKS inheritance. The governance commitment in CKS is architectural, not procedural; this is the first differentiation axis §5.2 names.

4. What OIDA inheritance is not, distinguished from four adjacent frames

OIDA's two primitives are commonly conflated with content from adjacent prior-art frames. The conflations are coherent on their own terms but produce inheritance attributions the source paper does not defend, and the standalone treatment requires distinguishing the OIDA inheritance from each.

Not paraconsistent logic. Paraconsistent logic is a family of consequence relations that are not explosive — that tolerate inconsistency without trivializing the theory. The Logics of Formal Inconsistency tradition internalizes consistency and inconsistency as object-language notions. The source paper §5.1 is explicit that paraconsistent logic supplies the *philosophical warrant* for treating contradictions as rational substrate content, not the *architecture* that operationalizes that stance. OIDA's signed contradiction edges are an architectural primitive — schema presence for a contradiction relationship — and the inheritance is at the architectural-primitive level, not the consequence-relation-semantics level. An implementation that inherits paraconsistent semantics without the schema-presence primitive has inherited from a different lineage than OIDA.

Not Dung-style abstract argumentation. Argumentation frameworks model pro and contra positions as graph nodes with attack and support edges, with multiple candidate stances over inconsistent information coexisting as extensions. The structural vocabulary is close to OIDA's at the surface, but the architectural commitments differ: argumentation edges name attack and support relations between *arguments* in an abstract argumentation graph; OIDA's signed contradiction edges name *propositional contradictions* between Knowledge Objects holding propositional content. The source paper §5.1

distinguishes these explicitly, and the OIDA inheritance is in the propositional-content frame.

Not detect-resolve-forget conflict pipelines. Conventional conflict-handling architectures detect contradictions, resolve them at runtime, and discard the contradiction record once resolved. OIDA's signed contradiction edges persist as substrate content regardless of resolution status; resolution does not erase the relationship record. The architectural difference is the persistence of the relationship as substrate state, and the inheritance is the persistence-as-substrate-state commitment, not the runtime-resolution algorithm.

Not pending-resolution treatment of unknowns. Many systems treat missing knowledge as pending-resolution state — an unknown that exists transiently until resolved. OIDA's QUESTION-as-modeled-ignorance is not pending-resolution: the QUESTION is a present substrate object that can persist indefinitely if no resolution is reached, and its addressability does not depend on its resolvability. The inheritance is the substrate-presence commitment for missing knowledge, not the workflow commitment that unknowns will be resolved.

5. Why OIDA inheritance is load-bearing for downstream commitments

The two-primitive OIDA inheritance carries weight in several downstream CKS commitments.

Conflict-as-first-class. The architectural promotion of the contradiction relationship to substrate state traces specifically to OIDA's signed contradiction edges. CKS's architectural surplus on the conflict axis is the combination of substrate-level preservation, provenance attached to the conflict itself, re-addressability across sessions, deferral as terminal outcome, and operation under the human-governance commitments — combination, not the representational primitive.

Substrate-as-source-of-truth. One of the categories of state that must live in the substrate is *active contradictions* — first-class addressable substrate objects with their own identity and provenance. That category-content commitment traces back through conflict-as-first-class to the OIDA-inherited relationship primitive.

Architectural-difference-versus-feature-addition. The claim that CKS is architecturally different from OIDA on the multi-human axis, not OIDA plus multi-human, requires the OIDA inheritance to be specified narrowly enough that the three differentiation axes (governance, role/authority, AI-as-mediator) are visible as architectural differences rather than gradations of inherited posture. A two-primitive inheritance bounded by three explicit differentiations is what supports the claim; a broader characterization would push CKS toward looking like an OIDA variant.

Two-axis extension structure. OIDA contributes inheritance on the conflict-and-missing-knowledge axis at single-human scope. The multi-human axis is silent in OIDA. CKS extends the inherited primitives along the multi-human axis under the architectural-governance commitment, and the two-axis structure is what makes the result recognizable as a coherent extension of inheritance rather than an addition of features.

6. Failure modes that mis-attribute OIDA inheritance

Implementations and analyses positioning CKS against OIDA can fail the inheritance commitment in several specific ways.

Over-claiming. Treating OIDA as the prior-art source for commitments the source paper attributes elsewhere — typed structured knowledge units (KO/Zahn-Chana per §5.2 and §8.2), provenance-field vocabulary (PROV-O / EP-Plan per §3.1 and §8.2), path-retraceability vocabulary (Rajabi & Kafaie per §3.1). Over-claiming weakens the prior-art posture by inviting OIDA-author contestation grounded in the source paper itself.

Under-claiming. Treating signed contradiction edges or QUESTION-as-modeled-ignorance as fresh in CKS when the source paper §5.2 attributes them as adjacent precedent. Under-claiming weakens credibility once reviewers identify the unacknowledged inheritance and undermines the architectural-difference claim, which depends on the inheritance being acknowledged.

Conflating with KO inheritance. Treating typed structured units as OIDA-inherited (it is KO/Zahn-Chana per source paper) or treating signed contradiction edges as KO-inherited (it is OIDA per source paper). The two prior-art sources contribute distinct architectural elements, and conflating them obscures the inheritance specificity that the architectural-difference claim relies on.

Conflating with philosophical-stance lineages. Treating paraconsistent-logic stance or Dung-argumentation structural vocabulary as OIDA inheritance. OIDA contributes the architectural primitive; the philosophical warrant comes from different lineages, and the source paper §5.1 separates them deliberately.

Treating OIDA inheritance as full-architectural-posture inheritance. Treating CKS's full architectural posture as OIDA-inherited and CKS as "OIDA plus multi-human." The source paper explicitly forecloses this misreading at §5.2 and again at §9.4: CKS is architecturally different from OIDA on the multi-human axis, not OIDA plus multi-human. The standalone OIDA-inheritance treatment is what makes the foreclosure precise.

Treating OIDA inheritance as implementational rather than architectural. Requiring specific OIDA codebases, libraries, or runtime components as a condition of inheritance. The architectural inheritance is at the pattern level: a CKS implementation written from scratch that carries signed contradiction edges and QUESTION-as-modeled-ignorance as substrate primitives instantiates the inheritance regardless of code lineage with OIDA implementations.

Selective inheritance. Inheriting signed contradiction edges without QUESTION-as-modeled-ignorance, or vice versa. The two primitives are paired analogues in §5.2's treatment; selective inheritance produces architectures that handle one axis but not the other and does not stand in the inheritance relationship the source paper defends.

7. Operational test

A system instantiates the OIDA-inheritance commitment in the architectural sense the source paper §5.2 defends if and only if all of the following are true.

1. The system carries signed contradiction edges as substrate-resident relationships between contradicting content elements, with schema presence (the contradiction edge has its own type), bidirectional addressing (the relationship references both contradicting elements), and persistence across sessions (the relationship survives session boundaries as substrate state, regardless of resolution status).
2. The system carries QUESTION-as-modeled-ignorance — explicit substrate-resident representation of missing knowledge as a present substrate object, addressable across sessions, distinguishable from absence-of-content, and persisting independently of resolvability.
3. Both primitives are inherited jointly; the system does not satisfy (1) without (2) or (2) without (1) and claim OIDA inheritance.
4. The inheritance is anchored to the §5.2 attribution; the system does not import OIDA's specific epistemic-class typology, importance-score-with-decay mechanism, or Knowledge Gravity Engine maintenance pattern as a condition of the inheritance, and does not claim OIDA inheritance for typed structured units, provenance-field vocabulary, or path-retraceability vocabulary, which the source paper attributes to other prior-art sources.
5. The three differentiations the source paper §5.2 names are preserved and explicit: the substrate is human-governed in the authority sense, encodes formal role and authority semantics at the cell level, and operates AI as substrate mediator rather than substrate client.
6. The inheritance is at the architectural-pattern level, not the implementational level; the system instantiates the two primitives and preserves the three differentiations regardless of code lineage with OIDA implementations.

A system that fails any of (1)–(6) does not stand in the OIDA-inheritance relationship the source paper §5.2 defends. A system that fails (1)–(3) but otherwise has OIDA-shaped features may be using OIDA-influenced primitives without instantiating the inheritance commitment in the architectural sense; a system that fails (5) is OIDA-shaped at the primitive level but not CKS-coherent at the architectural-posture level; a system that fails (4) is over-attributing inheritance and weakening the prior-art posture.

8. Why naming OIDA inheritance as standalone matters

Implementations and analyses positioning CKS against OIDA drift in two characteristic directions, and the standalone treatment is what resists each.

The first drift is over-attribution: presenting CKS as OIDA-with-a-governance-layer or OIDA-with-multi-human, by attributing to OIDA architectural content the source paper does not. Over-attribution weakens the architectural-difference claim and invites contestation by the OIDA authors on the basis that the source paper's actual attribution is narrower.

The second drift is under-attribution: presenting signed contradiction edges or modeled ignorance as fresh CKS contributions when the source paper acknowledges them as adjacent precedent. Under-attribution weakens credibility once reviewers identify the unacknowledged inheritance and makes the architectural-difference claim harder to defend, since the difference depends on the inheritance being explicitly bounded.

Naming the OIDA inheritance as a standalone architectural commitment — two primitives, three differentiations, seven failure modes, an operational test — gives downstream readers a precise specification that resists both drifts. Together with the parallel KO-inheritance treatment, the two-axis-extension-structure treatment, the architectural-difference-versus-feature-addition treatment, and the multi-human operational-requirements treatment, the standalone OIDA-inheritance note completes the operational decomposition of the parent foundational commitment and makes the source paper's positioning against OIDA defensible at the level of derivation that defensive publication requires.

Subsequent work that adopts the CKS pattern, extends it, composes it with adjacent patterns, or argues against it should treat OIDA inheritance as formalized here. Subsequent work that uses the inheritance relationship differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

Companion notes

Li, W. (2026). *Inheritance and Extension: How CKS Builds on Knowledge Objects and OIDA Along the Multi-Human Axis*. 27 April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *Authority, Not Labor: A Precise Definition of "Human-Governed" in the Coordination Knowledge Substrate Pattern*. 24 April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *What CKS Inherits from OIDA Alone: Signed Contradiction Edges and Modeled Ignorance as Standalone Architectural Primitives in the Coordination Knowledge Substrate Pattern*. 5 May 2026. ORCID: 0009-0004-8065-3235.